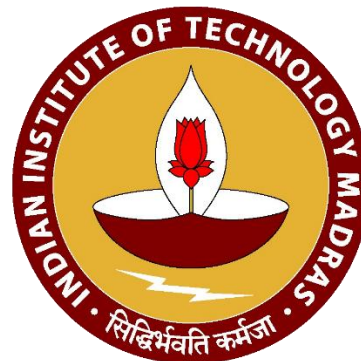


Python for Data Science

Web M. Tech Course DA5301W

Modules 6&7: Pandas

Dr. P.S Jayadev



Contents

- Lambda Functions
- Intro to Pandas
- Building Blocks of Pandas
- Selecting, Filtering and Updating DataFrames
- Views, Copies and Inplace Operations
- Grouping and Aggregation
- Handling Categorical and Missing Variables
- File IO Operations

Lambda Functions

- A lambda is a short, one-line function without a name and referred to as anonymous function
- Defined using the keyword **lambda**, followed by parameters and expression
 - Eg: `lambda x: x**2`
- The expression after `:` is automatically returned — no `return` keyword needed
- Commonly used to create a conditional rule without defining a full function
 - Eg: `nums = [3, 5, 8, 10]`
 - `list(filter(lambda x: x % 2 == 0, nums))`
- Can be used along with `if/else` condition and for loops as well
- **Note:** Use lambda functions only for inline one-time transformations
- Widely used in feature creation, aggregation, and preprocessing steps of machine learning

Pandas: Data Scientist's Workhorse

- Built on top of NumPy specifically to handle tabular data easily
- **Ideal for:**
 - Reading data (CSV / Excel / JSON)
 - Cleaning, transforming, aggregating, reshaping tabular data
 - Quick statistical analysis
- Optimised for fast and vectorized operations
- Integrates smoothly with popular python data science libraries such as numpy, matplotlib, scipy, scikit-learn, etc.
- Tip: Recommended to import pandas as **pd** (community standard)

Building Blocks of Pandas

- Dataframe, Series and Index are most important pandas specific objects (classes in pandas package)

Index

- Axis labels used to align data across columns
- **Analogy:** Row index or Primary key
- Pandas automatically assigns integer index (RangeIndex) but custom names can be given (strings, datetimes, categories, etc.)
- Immutable object and can be given a name

Series

- 1-D labeled array → holds one column of data (any dtype)
- **Analogy:** Excel or SQL column

DataFrame

- 2-D labeled table → collection of Series sharing an index
- **Analogy:** Excel sheet or SQL table
- Note: Assigning datatype to a column is generally more efficient (memory and speed) – Default is 'object' and not efficient

Selecting/Filtering DataFrame by Labels

- `.loc[]` selects data by **labels** — i.e., by the names of rows/columns
- **Syntax:** `df.loc[row_labels, column_labels]`
- `.loc` accepts:
 - Single label → `df.loc["row1", "col1"]`
 - List of labels → `df.loc[["row1", "row2"], ["col1", "col3"]]`
 - Label range (inclusive) → `df.loc["row1": "row3"]`
 - Boolean conditions → `df.loc[df["col2"] > 80, ["col1", "col2"]]`
- **Tip:** Specific both row and column labels for clarity

Selecting/Filtering DataFrame by Position

- `.iloc[]` selects data by **positions** like python lists
- **Syntax:** `df.iloc[row_positions, column_positions]`
- `.iloc` accepts:
 - Single integer → `df.iloc[0, 1]`
 - List of integers → `df.iloc[[0, 2], [1, 2]]`
 - Slice ranges (end excluded) → `df.iloc[0:3, 0:2]`
 - Boolean arrays → `df.iloc[[True, False, True, False], :]`
- Use `.iloc[]` when position is known but labels are dynamic or missing

Selecting/Filtering using query

- You use `.query()` to conditionally filter rows in a DataFrame when:
 - You want cleaner, more readable code than chained indexing
 - You need complex logical conditions involving multiple columns
 - You want to avoid repetitive `df[]` syntax
 - You want to dynamical filter based on the values of a variable
- Note: Do not use query in following scenarios:
 - When column names have spaces or special characters
 - When performance (speed) is critical

Views, Copies and Inplace Operations

Copying Dataframes

- When you assign one DataFrame to another (e.g. `df1 = df`), pandas does not create a new copy
 - It just creates another reference to the same object
 - Changes in one will affect the other unless you explicitly copy
- **Proper copy:** `df1 = df.copy()`
- When you “copy” a DataFrame, you can do:
 - **Shallow copy:** Reuse the same memory for the data while creating a new object
 - **Deep copy:** Duplicate the data completely (new object and new memory)
- **Shallow copy** saves memory and time when working with very large DataFrames where you only need to modify metadata (row/column labels)
- **Deep copy** ensures complete independence for transformations, modeling, or exporting data

Views in Dataframes

- A view is a reference to a portion of the original dataframe in memory
- Modifying a view directly changes the original DataFrame or Series because they share the same underlying data
- Views are returned in following scenarios:
 - Column selection
 - Selecting or filtering using loc or iloc
- When views of a dataframe are assigned to a new variable, then it is just a reference to the portal of the original dataframe
- **Best Practices:**
 - Always assume views are possible when slicing or selecting columns
 - If you plan to modify the result or create a new dataframe, then explicitly call `.copy()`
 - `_is_view` can be used to check if the returned value is view or not

Inplace Operations in Dataframes

- Inplace operation refers to modifying an existing dataframe without returning a new object
- Many pandas methods offer an `inplace=True` parameter
- **Note:** Does NOT happen at memory level and pandas creates a new object and reassigns it to same variable internally
- **When to use:**
 - When writing data cleaning pipelines, and you don't need the original data or multiple versions of it
 - When memory is not a major concern, and you want concise code
- **When not to use:**
 - When you may need the original DataFrame later for comparison or rollback
 - When debugging, inplace operations can make tracking harder

Assignment or Update Operations in Pandas

Setting/Updating Values in DataFrame

- You can set value to a single cell, multiple cells at once, entire column at once
- You can set or update values using:
 - Label-based assignment with `.loc[]` or `.at[]`
 - Position-based assignment with `.iloc[]` or `.iat[]`
 - Conditional assignment
 - Vectorised assignment with broadcasting if necessary
- Tips:
 - Vectorised assignment should be always preferred over python loops
 - Label based updates should be preferred when the same code will be used again

SettingWithCopyWarning in pandas

- **Issue:** Pandas sometimes returns a view and sometimes a copy and the behavior isn't always predictable
- **SettingWithCopyWarning** is shown when pandas is unsure whether your assignment will modify the original DataFrame or just a copy
- Eg: `df[df["A"] > 5]["B"] = 0` – chained indexing
 - `df[df["A"] > 5]` could be a view or a copy
- **Tip:** Better to avoid operations which result in this warning

Grouping and Aggregation

Splitting Data into Groups

- The `groupby()` operation allows one to:
 - **Split** data into groups based on one or more keys
 - **Apply** a function (aggregation, transformation, or filter)
 - **Combine** the results into a new DataFrame
- Grouping keys could be one or more column names, index if it is a string or category, external arrays, etc.
- **Aggregation Functions:**
 - Grouped data can be aggregated using built-in functions (mean, sum, count, min, max, etc.)
 - Multiple aggregations per group are possible
 - Custom functions or lambda can be used for aggregation

Transform and Filter in GroupBy

- Transformation is broadcasting group results back to each row
 - `transform()` preserves the original row structure
 - Useful for creating group-level statistics for every row
- Filtering refers to keeping or dropping entire groups based on a condition
- It takes a function that gets each group as a mini DataFrame and returns a Boolean (True/False)

Operation	Output Shape	Purpose
<code>agg()</code>	One row per group	Summarize
<code>transform()</code>	Same shape as original	Enrich / add derived columns
<code>filter()</code>	Subset of original	Select / exclude groups

Handling Categories and Missing Values

Handling Categories in DataFrames

- A special datatype in pandas for variables that take categorical values (Eg: Gender, region, department, etc.)
- Efficient for:
 - Repeated string values (saves memory)
 - Faster grouping, sorting, and filtering
- Always define **category order** when comparisons are meaningful
 - Eg: Low, Medium, High
- When assigning new values not in `.categories`, first use `.cat.add_categories()`, otherwise it results in an error

Handling Missing Values in Dataframes

- Missing values in Pandas are represented by:
 - NaN (Not a Number) for numeric object
 - NaT (Not a Time) for datetime object
 - None for object/string columns
- Pandas treats all of them as missing indicators
- Standard methods exist to detect missing values in DataFrames
 - `df.isna()` and `df.notna()`
 - `df.info()` gives a quick overview of missing data
- **Note:** Empty strings ("") or placeholders like 'NA' are not be auto-detected as missing
- **Treating Missing Values:** `df.dropna()`, `df.fillna()`

File IO Operations in Pandas

File Input/Output in Pandas

- Pandas can read data from multiple sources: csv, excel, json, sql
- `df = pd.read_csv("data.csv")`, `df = pd.read_excel("data.xlsx")`
- Key features:
 - Possible to select specific columns while reading data
 - Possible to read limited rows for testing
 - Enforce data types for specific columns and raise errors when data type does not match
 - Parse date columns as datetime datatype
 - Possible to select specific sheets and header column when reading from xlsx format

File Input/Output in Pandas

- Pandas can write data to files of different types: csv, excel, json, sql
- `df.to_csv("data.csv")`, `df.to_excel("data.xlsx")`
- Key features:
 - Possible to choose whether to include the index or not through 'index=True/False' parameter
 - Possible to export select columns by passing a list of columns via columns parameter
 - Write to a target sheet when writing in xlsx format via sheet_name parameter
 - Option to write multiple dataframes to multiple sheets in a xlsx format